

Plotly Dashboard

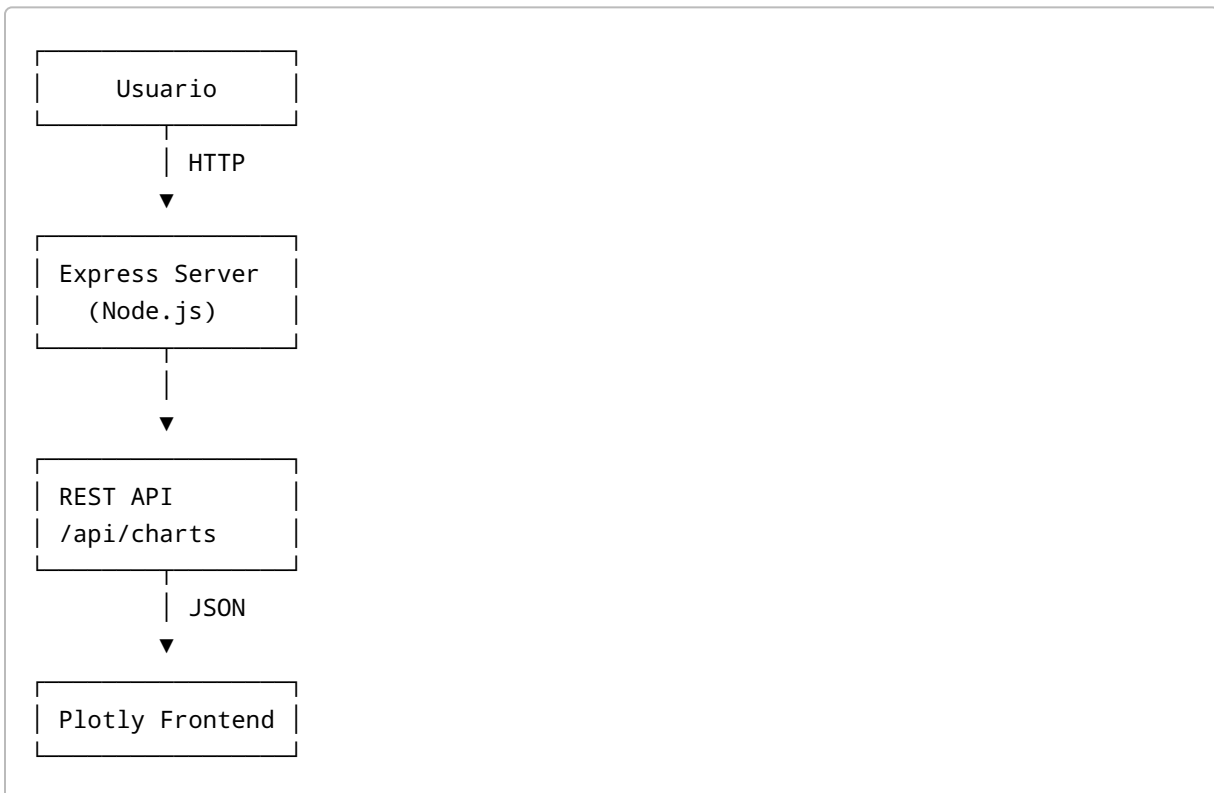
Descripción

Plotly Dashboard es una aplicación web desarrollada con Node.js y Express que permite visualizar información mediante gráficos interactivos generados con Plotly.js.

El proyecto implementa prácticas DevOps mediante:

- Control de versiones con Git y GitHub.
- Integración Continua (CI) utilizando GitHub Actions.
- Pruebas unitarias con Jest y Supertest.
- Contenerización mediante Docker.
- Entrega Continua (CD) utilizando Jenkins.
- Publicación de imágenes en Docker Hub.

Arquitectura



Tecnologías Utilizadas

Tecnología	Versión
Node.js	22.x
Express	5.x
Plotly.js	3.6.0
Jest	Última
Supertest	Última
Docker	28+
Jenkins	LTS
GitHub Actions	Última

Estructura del Proyecto

```
plotly-dashboard/
├── .github/
│   └── workflows/
│       └── ci.yml
├── public/
│   ├── index.html
│   ├── style.css
│   └── app.js
├── tests/
│   └── server.test.js
├── Dockerfile
├── Jenkinsfile
├── package.json
├── package-lock.json
├── server.js
├── .gitignore
└── README.md
```

Instalación Local

Clonar el repositorio

```
git clone https://github.com/sergioing1000/plotly-dashboard.git  
  
cd plotly-dashboard
```

Instalar dependencias

```
npm install
```

Ejecutar la aplicación

```
npm start
```

La aplicación estará disponible en:

```
http://localhost:3000
```

API REST

Obtener datos de los gráficos

Endpoint

```
GET /api/charts
```

Respuesta

```
{  
  "sales": {  
    "labels": ["January", "February", "March"],  
    "values": [150, 200, 180]
```

```
}  
}
```

Pruebas Unitarias

Las pruebas utilizan:

- Jest
- Supertest

Ejecutar pruebas

```
npm test
```

Cobertura

```
npm run coverage
```

Integración Continua (CI)

La integración continua está implementada mediante GitHub Actions.

Flujo

1. Checkout del código.
2. Instalación de dependencias.
3. Ejecución de pruebas unitarias.
4. Generación de reportes de cobertura.
5. Validación del build.

Archivo:

```
.github/workflows/ci.yml
```

Ejecución Manual

```
git add .  
git commit -m "Nueva funcionalidad"  
git push origin main
```

El workflow se ejecutará automáticamente.

Docker

Construcción de Imagen

```
docker build -t plotly-dashboard .
```

Ejecución del Contenedor

```
docker run -p 3000:3000 plotly-dashboard
```

Verificar

```
http://localhost:3000
```

Dockerfile

```
FROM node:22-alpine  
  
WORKDIR /app  
  
COPY package*.json ./  
  
RUN npm install  
  
COPY . .  
  
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

Entrega Continua (CD)

La entrega continua se implementa mediante Jenkins.

Flujo CD

1. Clonar repositorio.
2. Construir imagen Docker.
3. Ejecutar pruebas.
4. Publicar imagen en Docker Hub.
5. Desplegar nueva versión.

Jenkinsfile

```
pipeline {
  agent any

  environment {
    IMAGE_NAME = "plotly-dashboard"
    DOCKER_USER = "usuarioDockerHub"
  }

  stages {
    stage('Clone Repository') {
      steps {
        git 'https://github.com/sergioing1000/plotly-dashboard.git'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }

    stage('Run Tests') {
      steps {
```

```

        sh 'npm test'
    }
}

stage('Build Docker Image') {
    steps {
        sh 'docker build -t $DOCKER_USER/$IMAGE_NAME:latest .'
    }
}

stage('Push Docker Image') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER_NAME',
                passwordVariable: 'DOCKER_PASSWORD'
            )
        ]) {
            sh '''
password-stdin
                echo $DOCKER_PASSWORD | docker login -u $DOCKER_USER_NAME --

                docker push $DOCKER_USER/$IMAGE_NAME:latest
            '''
        }
    }
}
}
}
}
}

```

Variables de Entorno

Variable	Descripción
PORT	Puerto del servidor
NODE_ENV	Ambiente de ejecución

Ejemplo:

```

PORT=3000
NODE_ENV=production

```

Seguridad

- Dependencias gestionadas mediante npm.
- Escaneo de vulnerabilidades con:

```
npm audit
```

- Imágenes Docker basadas en Alpine Linux.
 - Uso de credenciales seguras en Jenkins.
-

Monitoreo

Se recomienda integrar:

- Prometheus
- Grafana
- Docker Health Checks

Ejemplo:

```
HEALTHCHECK CMD wget --spider http://localhost:3000 || exit 1
```

Autor

Sergio Cruz

Proyecto académico para la implementación de prácticas DevOps, Integración Continua (CI) y Entrega Continua (CD).